



L'ÉCOLE
DES INGÉNIEURS
DU NUMÉRIQUE

Projet de cryptographie

Sécurisation d'une application de messagerie instantanée

Groupe : TETE BENISSAN Nerista, NAPOLETANO Cyndelle

Date : 15/04/2026

1. Introduction	2
2. Test de l'application initiale	3
2.1 Compilation et tests	3
2.1.1 Principe de ROT13	3
2.1.2 Captures d'écrans	4
2.2 Implémentation d'une attaque sur rot13	5
2.2.1 Analyse	5
2.2.2 Capture d'écran	5
3. Ajout du chiffrement symétrique	6
3.1 Modification du client avec mot de passe	6
3.2 Dérivation d'une clé symétrique AES (algorithme de hachage)	6
3.3 Implémentation du chiffrement AES et changement de convention	7
3.4 Attaque de modification de message	7
3.4.1 Explications	7
3.4.2 Captures d'écrans de l'attaque	8
3.4.3 Autres configurations possibles	9
4. Ajout de l'intégrité des messages	10
5. Ajout de l'établissement de clé de session	11
5.1 Utilisation d'une clé ECDH éphémère	11
5.2 Implémentation d'une attaque MitM	12
6. Ajout de la signature des clés publiques ECDH	14
6.1 Génération d'une paire de clé ECDSA long terme	14
6.2 Modification de la phase de handshake	14
6.3 Implémentation d'une attaque MitM	15
7. Certification des clés	17
7.1 Génération de la PKI	17
7.2 Mise à jour du protocole de handshake	17
7.3 Détection du MitM	17
8. Attaques résiduelles	20
8.1 Attaques encore possibles	20
8.2 Mécanisme de protection	22
8.3 Perfect Forward Secrecy	22
9. Difficultés rencontrées	23
10. Améliorations possibles du projet	24
11. Ressources utilisées	25
12. Conclusion	26

1. Introduction

Ce projet a pour objectif de sécuriser progressivement une application de messagerie instantanée écrite en Java. Deux clients communiquent via un serveur de relais, lequel peut jouer le rôle d'un attaquant Man-in-the-Middle (MitM). L'ensemble du système est simulé localement sur une seule machine.

L'approche adoptée est incrémentale : chaque étape introduit un mécanisme cryptographique supplémentaire et en démontre l'utilité face à des attaques concrètes. Le projet couvre ainsi quatre propriétés fondamentales : la confidentialité (AES-GCM), l'intégrité (tag d'authentification GCM), l'authentification (ECDSA et certificats X.509), et la résistance au replay et à la suppression (numéros de séquence).

Toutes les primitives cryptographiques utilisées ciblent un niveau de sécurité de 128 bits, conformément aux recommandations du NIST. Les algorithmes retenus sont : AES-128/GCM pour le chiffrement symétrique, ECDH sur la courbe secp256r1 pour l'échange de clés, SHA256withECDSA pour les signatures numériques et SHA-256 pour la dérivation de clés.

Fichiers disponibles pour réaliser cette application :

```
PS C:\Users\aline\Documents\ISEN4\Cryptographie\projet\messagerie_instantanee_cryptographie\CryptoChat> tree /F
Structure du dossier pour le volume Acer
Le numéro de série du volume est 500B-092E
C:..
  Client$MessageReceiver.class
  Client.class
  Client.java
  ClientHandler.class
  Interceptor.class
  Interceptor.java
  Server.class
  Server.java
  ServerInterceptor.class
  ServerInterceptor.java
  start.bat

Aucun sous-dossier existant
```

2. Test de l'application initiale

2.1 Compilation et tests

2.1.1 Principe de ROT13

Le ROT13 (*rotate by 13 places*) est un cas particulier du chiffrement de César, un algorithme très simple de **chiffrement par substitution**. Il consiste à décaler chaque lettre de l'alphabet de 13 positions. Une particularité de ROT13 est que l'opération de chiffrement et de déchiffrement est identique : appliquer deux fois la transformation permet de retrouver le texte original.

Cependant, ROT13 ne respecte pas le principe de Kerckhoffs. En effet, si un adversaire connaît ou devine l'algorithme utilisé, il peut immédiatement retrouver le message en clair, car aucune clé secrète n'est utilisée.

Dans la plupart des implémentations, seules les lettres non accentuées sont transformées, tandis que les autres caractères (lettres accentuées, chiffres, ponctuation) restent inchangés. Dans notre application, nous avons remarqué que les accents sont supprimés et les chiffres sont transmis en clair.

```
cyndelle tu es là?  
[Interceptor] Encrypting message: cyndelle tu es l??
```

```
[Server -> Client 1]: zba ahzreb rfg 085527894126
```

Toutefois, une variante appelée ROT47 permet de transformer un ensemble plus large de caractères, incluant lettres, chiffres et symboles, en se basant sur les codes ASCII.

2.1.2 Captures d'écrans

```
C:\Windows\System32\cmd.exe

Starting server and clients...

[1/3] Launching Server...
[2/3] Launching Client 1...
[3/3] Launching Client 2...

=====
All windows launched successfully!
=====

You should now see 3 windows:
- Server window (accepting connections)
- Client 1 window (ready to chat)
- Client 2 window (ready to chat)

To chat:
1. Click on a client window
2. Type your message
3. Press Enter to send

To exit:
- Type 'exit' in client windows
- Close the server window when done

=====

Appuyez sur une touche pour continuer...
```

```
Crypto Chat - Client 1 - java Client

Starting client ...
=== Crypto Chat Client ===
Connecting to server at localhost:8888...
Connected to server successfully!

Waiting for other client to connect...
Both clients connected!

--- Handshake Phase ---
[Interceptor] Starting handshake
[Interceptor] Handshake complete!
--- Handshake Complete ---

Chat session started!
Type your messages and press Enter to send.
Type 'exit' to quit.

[Interceptor] Decrypting message...
Other: salut
[Interceptor] Decrypting message...
Other: comment tu vas
bien et toi
[Interceptor] Encrypting message: bien et toi
You: bien et toi
[Interceptor] Decrypting message...
Other: pouik
```

```
Crypto Chat - Server - java Server

[Server] Honest relay mode
=== Crypto Chat Server ===
Starting server on port 8888...
Server started successfully!
Waiting for 2 clients to connect...

Client 1 connected from 127.0.0.1
Client 2 connected from 127.0.0.1

All clients connected! Chat session can begin.

[Server -> Client 1]: READY
[Server -> Client 2]: READY
Server is now relaying messages between clients...
[Client 2 -> Server]: fnyhg
Relaying from 2 to client 1 : fnyhg
[Server -> Client 1]: fnyhg
[Client 2 -> Server]: pbzzrag gh inf
Relaying from 2 to client 1 : pbzzrag gh inf
[Server -> Client 1]: pbzzrag gh inf
[Client 1 -> Server]: ovra rg gbv
Relaying from 1 to client 2 : ovra rg gbv
[Server -> Client 2]: ovra rg gbv
[Client 2 -> Server]: cbhvx
Relaying from 2 to client 1 : cbhvx
[Server -> Client 1]: cbhvx
```

```
Crypto Chat - Client 2 - java Client

Starting client ...
=== Crypto Chat Client ===
Connecting to server at localhost:8888...
Connected to server successfully!

Waiting for other client to connect...
Both clients connected!

--- Handshake Phase ---
[Interceptor] Starting handshake
[Interceptor] Handshake complete!
--- Handshake Complete ---

Chat session started!
Type your messages and press Enter to send.
Type 'exit' to quit.

salut
[Interceptor] Encrypting message: salut
You: salut
comment tu vas
[Interceptor] Encrypting message: comment tu vas
You: comment tu vas
[Interceptor] Decrypting message...
Other: bien et toi
pouik
[Interceptor] Encrypting message: pouik
You: pouik
```

2.2 Implémentation d'une attaque sur rot13

2.2.1 Analyse

Dans cette première version du projet, le chiffrement côté client repose sur ROT13. Or comme nous l'avons vu, ROT13 est une transformation déterministe, sans clé secrète et son inversion est identique à son application. Cela permet à un attaquant qui observe les messages chiffrés de retrouver immédiatement le texte en clair.

Pour simuler une attaque de type **Man-in-the-Middle**, nous utilisons le fait que tous les messages transitent par le serveur. Le serveur joue alors le rôle d'adversaire : il intercepte chaque message avant de le relayer au destinataire, puis applique ROT13 sur le message intercepté afin d'obtenir le contenu en clair. Enfin, il relaie le message original inchangé, ce qui permet à la conversation de continuer normalement sans alerter les utilisateurs.

Concrètement, l'attaque est implémentée dans `ServerInterceptor.onMessageRelay()`:

1. Le serveur affiche le message intercepté (encore "chiffré" en ROT13).
2. Il calcule le rot13 (message) pour le déchiffrer.
3. Il affiche le résultat en clair dans la console serveur.
4. Il renvoie le message original pour ne pas modifier le trafic.

Cette attaque démontre que ROT13 n'assure aucune confidentialité dès lors qu'un relais de transport (ici le serveur) est malveillant ou compromis.

2.2.2 Capture d'écran

```
Crypto Chat - Server - java Server
Client 2 connected from 127.0.0.1

All clients connected! Chat session can begin.

[Server -> Client 1]: READY
[Server -> Client 2]: READY
Server is now relaying messages between clients...
[Client 2 -> Server]: pbhpbh
Intercepted message (encrypted): pbhpbh
Decrypted message (cleartext): coucou
[Server -> Client 1]: pbhpbh
[Client 1 -> Server]: plaqryyr gh rf y??
Intercepted message (encrypted): plaqryyr gh rf y??
Decrypted message (cleartext): cyndelle tu es l??
[Server -> Client 2]: plaqryyr gh rf y??
[Client 2 -> Server]: bhv wr fhvf yn arevfgn
Intercepted message (encrypted): bhv wr fhvf yn arevfgn
Decrypted message (cleartext): oui je suis la nerista
[Server -> Client 1]: bhv wr fhvf yn arevfgn
[Client 2 -> Server]: *d
Intercepted message (encrypted): *d
Decrypted message (cleartext): *q
[Server -> Client 1]: *d
[Client 2 -> Server]: dhbv
Intercepted message (encrypted): dhbv
Decrypted message (cleartext): quoi
[Server -> Client 1]: dhbv
[Client 2 -> Server]: zba ahzreb rfg 085527894126
Intercepted message (encrypted): zba ahzreb rfg 085527894126
Decrypted message (cleartext): mon numero est 085527894126
[Server -> Client 1]: zba ahzreb rfg 085527894126
[Client 2 -> Server]: zbv
Intercepted message (encrypted): zbv
Decrypted message (cleartext): moi
[Server -> Client 1]: zbv
```

3. Ajout du chiffrement symétrique

3.1 Modification du client avec mot de passe

Dans cette étape, le client est modifié afin de lire un mot de passe fourni en argument lors du lancement du programme. Ce mot de passe sera utilisé pour dériver une clé de chiffrement commune entre les deux participants.

Le lancement du client se fait désormais de la manière suivante :

```
java Client secret123
```

Le mot de passe est récupéré dans les arguments de la ligne de commande (args[]) et transmis au constructeur de la classe Interceptor. Cette modification permet de forcer l'utilisateur à fournir un mot de passe lors de l'établissement de la connexion.

Les deux clients doivent utiliser le même mot de passe afin de pouvoir dériver la même clé de chiffrement. Si les mots de passe sont différents, les messages chiffrés ne pourront pas être correctement déchiffrés par l'autre participant.

Cette modification constitue la base de la mise en place d'un chiffrement symétrique pour les messages échangés dans le chat.

3.2 Dérivation d'une clé symétrique AES (algorithme de hachage)

Un mot de passe ne peut pas être utilisé directement comme clé AES. En effet, AES nécessite une clé binaire de taille précise (128, 192 ou 256 bits). Pour transformer le mot de passe en clé cryptographique utilisable, on applique une fonction de hachage cryptographique.

Dans notre implémentation, la fonction **SHA-256** est utilisée.

Le processus de dérivation de clé est le suivant :

```
mdp ⇒ SHA-256 ⇒ hash de 256 bits (32 octets) ⇒ 16 premiers octets ⇒ clé AES-128
```

La fonction SHA-256 produit un résultat de 256 bits (32 octets). Comme nous utilisons AES-128, qui requiert une clé de 128 bits (16 octets), seuls les 16 premiers octets du hash sont conservés.

Ces octets sont ensuite utilisés pour construire une clé AES qui pourra être utilisée par l'API cryptographique Java.

Cette technique permet de transformer un mot de passe en clé symétrique utilisable pour le chiffrement et le déchiffrement des messages.

3.3 Implémentation du chiffrement AES et changement de convention

```

tantanee_cryptographie\CryptoChat> exit
java Client secret123

Starting client ...
=== Crypto Chat Client ===
Connecting to server at localhost:8888...
Connected to server successfully!

Waiting for other client to connect...
Both clients connected!

--- Handshake Phase ---
[Interceptor] Starting handshake
[Interceptor] Handshake complete!
--- Handshake Complete ---

Chat session started!
Type your messages and press Enter to send.

tantanee_cryptographie\CryptoChat> java Client secret123

Starting client ...
=== Crypto Chat Client ===
Connecting to server at localhost:8888...
Connected to server successfully!

Waiting for other client to connect...
Both clients connected!

--- Handshake Phase ---
[Interceptor] Starting handshake
[Interceptor] Handshake complete!
--- Handshake Complete ---

Chat session started!
Type your messages and press Enter to send.

```

Maintenant que le mot de passe est établi et les changements effectués sur le code, on obtient ce workflow :

Lors de l'envoi :

```

message en clair
↓
chiffrement AES
↓
données binaires
↓
encodage Base64
↓
message transmis au serveur

```

Lors de la réception :

```

message Base64
↓
décodage Base64
↓
données chiffrées
↓
déchiffrement AES
↓
message en clair

```

Cette convention (AES + Base64) est conservée pour le reste du projet afin de faciliter la visualisation et le transport des messages chiffrés.

3.4 Attaque de modification de message

3.4.1 Explications

Dans cette étape, le serveur continue de jouer le rôle d'attaquant **Man-in-the-Middle**. Il peut intercepter les messages échangés entre les deux clients et tenter de les modifier avant de les relayer.

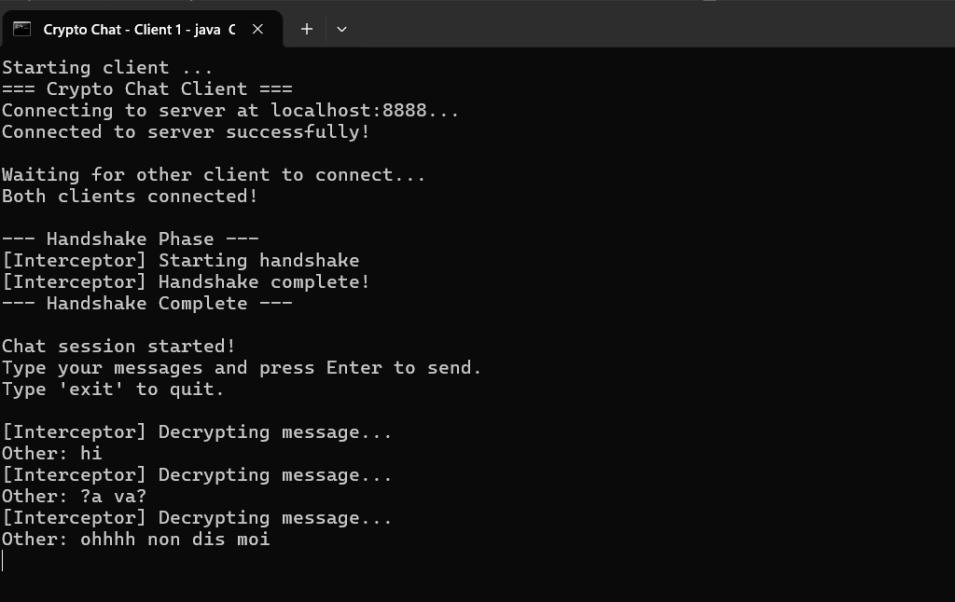
Lorsque les messages sont simplement chiffrés avec AES (sans mécanisme d'authentification), un attaquant peut modifier certaines parties du message chiffré. Selon le mode de chiffrement et le padding utilisé, cette modification peut parfois passer inaperçue.

Par exemple, certains modes de chiffrement peuvent produire un message déchiffré incorrect sans provoquer d'erreur ou le padding peut parfois rester valide malgré une modification du texte chiffré.

Dans ces situations, le destinataire reçoit un message modifié sans être averti de l'attaque. Cela montre que le chiffrement seul ne garantit pas l'intégrité des messages. Pour conclure, un mécanisme supplémentaire est nécessaire pour détecter toute modification du texte chiffré.

3.4.2 Captures d'écrans de l'attaque

Point de vue du client 1 lors de l'attaque :



```
Crypto Chat - Client 1 - java
Starting client ...
=== Crypto Chat Client ===
Connecting to server at localhost:8888...
Connected to server successfully!

Waiting for other client to connect...
Both clients connected!

--- Handshake Phase ---
[Interceptor] Starting handshake
[Interceptor] Handshake complete!
--- Handshake Complete ---

Chat session started!
Type your messages and press Enter to send.
Type 'exit' to quit.

[Interceptor] Decrypting message...
Other: hi
[Interceptor] Decrypting message...
Other: ?a va?
[Interceptor] Decrypting message...
Other: ohhhh non dis moi
```

Point de vue du serveur (Man-in-the-middle) :

```
Crypto Chat - Server - java Se × + ▾

[Server] MITM modification mode
=== Crypto Chat Server ===
Starting server on port 8888...
Server started successfully!
Waiting for 2 clients to connect...

Client 1 connected from 127.0.0.1
Client 2 connected from 127.0.0.1

All clients connected! Chat session can begin.

[Server -> Client 1]: READY
[Server -> Client 2]: READY
Server is now relaying messages between clients...
[Client 2 -> Server]: rCON0KdeGg4MKZc2ZrB06KME
[MITM] Message intercepté (Base64) : rCON0KdeGg4MKZc2ZrB06KME
[Server -> Client 1]: rCON0KdeGg4MKZc2ZrB06KME
[Client 2 -> Server]: +z30cJNFQHsCy0ms52+DEgguVhBMicz0
[MITM] Message intercepté (Base64) : +z30cJNFQHsCy0ms52+DEgguVhBMicz0
[Server -> Client 1]: +z30cJNFQHsCy0ms52+DEgguVhBMicz0
[Client 2 -> Server]: Zg6KHZh2E7RLllht2vJnVErG4qAxmPxIsQNqmVQe/9dB
[MITM] Message intercepté (Base64) : Zg6KHZh2E7RLllht2vJnVErG4qAxmPxIsQNqmVQe/9dB
[Server -> Client 1]: Zg6KHZh2E7RLllht2vJnVErG4qAxmPxIsQNqmVQe/9dB
```

3.4.3 Autres configurations possibles

On modifie le code `serverinterceptor.java`, mais dans le premier exemple que nous avons vu juste au-dessus, rien n'est détecté. Cependant, si on modifie certains éléments, cela peut également provoquer une erreur ou produire un message corrompu.

4. Ajout de l'intégrité des messages

A présent, nous avons remplacé le mode AES initialement utilisé par le mode AES-GCM (Galois/Counter Mode).

Contrairement aux modes classiques d'AES, AES-GCM ne se limite pas au chiffrement : il fournit également un mécanisme d'authentification des données.

Concrètement, en plus de chiffrer le message, AES-GCM génère un tag d'authentification (MAC) basé sur le contenu du message et une clé secrète. Ce tag est transmis avec le message chiffré.

Lors de la réception, le destinataire recalcule ce tag et le compare à celui reçu : si les deux correspondent, le message est considéré comme intègre et est déchiffré sinon, le message est rejeté.

Cela permet de détecter toute modification du message, même minime (un seul bit modifié suffit à invalider le tag).

Comme attendu dans l'étape 3.3 du projet, une attaque de modification de message (via le ServerInterceptor simulant un attaquant man-in-the-middle) est désormais détectée automatiquement par le serveur, qui refuse alors le message altéré :

```
Crypto Chat - Server - java Se X + v
[Server] MITM modification mode
=== Crypto Chat Server ===
Starting server on port 8888...
Server started successfully!
Waiting for 2 clients to connect...

Client 1 connected from 127.0.0.1
Client 2 connected from 127.0.0.1

All clients connected! Chat session can begin.

[Server -> Client 1]: READY
[Server -> Client 2]: READY
Server is now relaying messages between clients...
[Client 2 -> Server]: kiV4hG557jDAI/ata3wHUQB58qxGIyXBcisaD880TIJyE3SDxASYbypX
[MITM] Message intercepté (Base64) : kiV4hG557jDAI/ata3wHUQB58qxGIyXBcisaD880TIJyE3SDxASYbypX
[MITM] Un bit a été modifié à l'index 30.
[Server -> Client 1]: kiV4hG557jDAI/ata3wHUQB58qxGIyXBcisaD880TYJyE3SDxASYbypX
[Client 2 -> Server]: GYZaGQAmREi9YLZtP5r1BkIbKmiots0/5b0=
[MITM] Message intercepté (Base64) : GYZaGQAmREi9YLZtP5r1BkIbKmiots0/5b0=
[Server -> Client 1]: GYZaGQAmREi9YLZtP5r1BkIbKmiots0/5b0=
[Client 2 -> Server]: 02ETRtEq3XW6Yy71URWm5iac01pAWw==
[MITM] Message intercepté (Base64) : 02ETRtEq3XW6Yy71URWm5iac01pAWw==
[Server -> Client 1]: 02ETRtEq3XW6Yy71URWm5iac01pAWw==
```

Un message apparaît également du côté client :

```
Crypto Chat - Client 1 - java C x + v
Connecting to server at localhost:8888...
Connected to server successfully!

Waiting for other client to connect...
Both clients connected!

--- Handshake Phase ---
[Interceptor] Starting handshake
[Interceptor] Handshake complete!
--- Handshake Complete ---

Chat session started!
Type your messages and press Enter to send.
Type 'exit' to quit.

[Interceptor] Decrypting message (AES-GCM)...
Other: salut
oui dis moi
[Interceptor] Encrypting message (AES-GCM): oui dis moi
You: oui dis moi
non rien merci
[Interceptor] Encrypting message (AES-GCM): non rien merci
You: non rien merci
[Interceptor] Decrypting message (AES-GCM)...
Other: je tenproe
[Interceptor] Decrypting message (AES-GCM)...
Other: hum
[Interceptor] Decrypting message (AES-GCM)...
Other: [Decryption failed: Tag mismatch]
```

5. Ajout de l'établissement de clé de session

5.1 Utilisation d'une clé ECDH éphémère

Dans cette nouvelle partie, nous avons supprimé l'utilisation du mot de passe. Une clé ECDH éphémère générée à chaque nouvelle connexion au serveur sert désormais de nouveau point d'entrée à l'application. Voici à quoi ressemble la connexion du client à présent :

```
Crypto Chat - Client 1 - java C x + v
Starting client ...
=== Crypto Chat Client ===
Connecting to server at localhost:8888...
Connected to server successfully!

Waiting for other client to connect...
Both clients connected!

--- Handshake Phase ---
[Interceptor] Starting ECDH handshake
[Interceptor] Clé publique ECDH envoyée
[Interceptor] Clé publique de l'autre client reçue
[Interceptor] Clé de session dérivée, handshake terminé !
--- Handshake Complete ---

Chat session started!
Type your messages and press Enter to send.
Type 'exit' to quit.

[Interceptor] Decrypting message (AES-GCM)...
Other: ok viens ? 20h
non stp
[Interceptor] Encrypting message (AES-GCM): non stp
You: non stp
pas comme ca
[Interceptor] Encrypting message (AES-GCM): pas comme ca
You: pas comme ca
[Interceptor] Decrypting message (AES-GCM)...
Other: pourquoi?
```

```
Crypto Chat - Client 2 - java C x + v
Starting client ...
=== Crypto Chat Client ===
Connecting to server at localhost:8888...
Connected to server successfully!

Waiting for other client to connect...
Both clients connected!

--- Handshake Phase ---
[Interceptor] Starting ECDH handshake
[Interceptor] Clé publique ECDH envoyée
[Interceptor] Clé publique de l'autre client reçue
[Interceptor] Clé de session dérivée, handshake terminé !
--- Handshake Complete ---

Chat session started!
Type your messages and press Enter to send.
Type 'exit' to quit.

ok viens à 20h
[Interceptor] Encrypting message (AES-GCM): ok viens ? 20h
You: ok viens ? 20h
[Interceptor] Decrypting message (AES-GCM)...
Other: non stp
[Interceptor] Decrypting message (AES-GCM)...
Other: pas comme ca
pourquoi?
[Interceptor] Encrypting message (AES-GCM): pourquoi?
You: pourquoi?
```

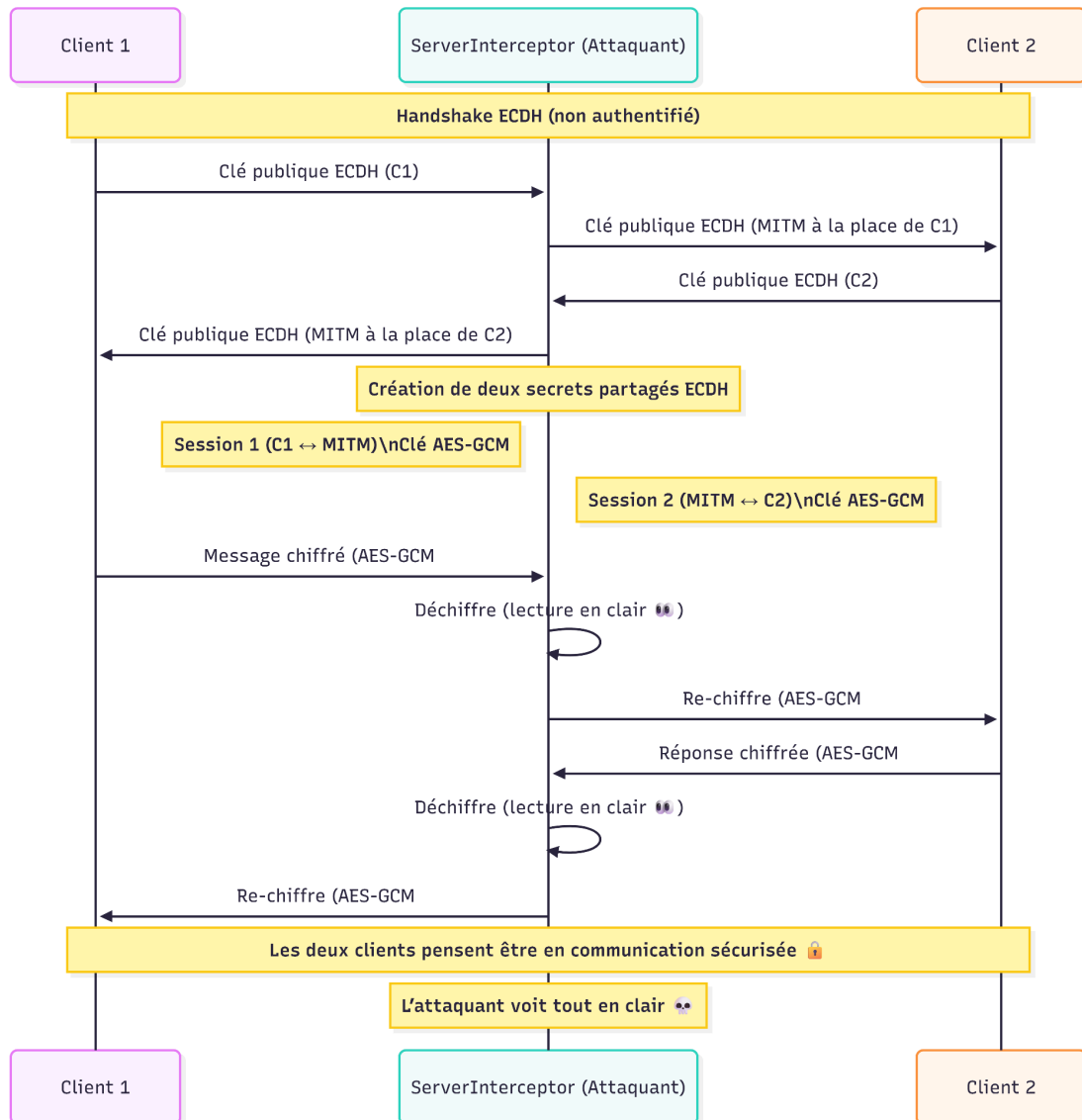
5.2 Implémentation d'une attaque MitM

Or, ce que les captures précédentes ne révèlent pas, c'est la présence du MitM, qui est mise en avant du côté du serveur :

```
Crypto Chat - Server - java Se x + v
[Server -> Client 1]: READY
[Server -> Client 2]: READY
Server is now relaying messages between clients...
[Client 2 -> Server]: MFkwEwYHkoZizj0CAQYIKoZizj0DAQcDQgAetlVnL4gGoeb575ktcpXjmbXbLKhEpQFSsDJRv0T/OTL4SrSaKIw8/+62BhvVb
hmRHvrXXbV/TsYk/FnSkZe+g==
[Client 1 -> Server]: MFkwEwYHkoZizj0CAQYIKoZizj0DAQcDQgAEp/7d1F2yh/bqbg0tEJTAWx3HBR4yLY9ltge4HwU93wONaZ6TkA2mojp+F9kuHv
ULJmHUqW1RjnX+yhBAFvDfSw==
[MITM] Interception clé publique ECDH du Client 2
[MITM] Clé de session K2 dérivée avec Client 2
[MITM] Clé MitM envoyée au Client 1 à la place de Client 2
[Server -> Client 1]: MFkwEwYHkoZizj0CAQYIKoZizj0DAQcDQgAEfn4i5oHg0h4jhG9Me12o7yuVEyQgnY7bk7C/gnG5iSD3tKzLqnjxcZM2ZHCseD
Ll88bbUdV17LTSWdI/Zqy4nQ==
[MITM] Interception clé publique ECDH du Client 1
[MITM] Clé de session K1 dérivée avec Client 1
[MITM] Clé MitM envoyée au Client 2 à la place de Client 1
[Server -> Client 2]: MFkwEwYHkoZizj0CAQYIKoZizj0DAQcDQgAEQcTwifqo8ZnY4gbutzJEIrVpFvQsh32GKp35c0o2gKFoLr9NZ3zcPXjjjiqrh8
xQhLG5UBTMD71TPYJM0odsWw==
[Client 2 -> Server]: zkkELailSORhqN6lQx89qNdb1tEKDewREzUq62F/WFxNoc1TC7xxTqciWR0=
[MITM] Message en clair intercepté : ok viens ? 20h
[Server -> Client 1]: rr9OJBpQztuqVZRctdEGoEsNfqkv/+lVRwBQ0e0gJzjUK/1wo0cKVCN0uqY=
[Client 1 -> Server]: 092d71WUg7MwZhr4NCuYneDzmlVCKPh5swqxdvnjtiEateI=
[MITM] Message en clair intercepté : non stp
[Server -> Client 2]: 1fLGqlG+dY1s9GeXTm80VULdnKTS93qLwUNkVQNgwf13vfA=
[Client 1 -> Server]: iY093lJTETUP48T5WMVRuxonaWiUXEW3hChQAJnFw1pX0v5L97Fh3g==
[MITM] Message en clair intercepté : pas comme ça
[Server -> Client 2]: XGiYW2RfMRZonRNhey7hZ3MoMVMZguH3adLF3BOCnfVSTbqWaaW0Dg==
[Client 2 -> Server]: JOFJiKMBMDUDZfjCS7FvsHy80j7uyrX2m/OP/m7AUe+o6fqzWA==
[MITM] Message en clair intercepté : pourquoi?
[Server -> Client 1]: k7HQLdLr7z0K6z+pXoYwQMUBfcuUwGr6x+NUF2JP9PvwDRcYlg==
```

Ainsi, malgré le chiffrement AES-GCM, une attaque Man-in-the-Middle reste possible dès lors que les clés publiques ECDH ne sont pas authentifiées. Le `ServerInterceptor` génère ses propres paires de clés ECDH, intercepte les clés éphémères des deux clients et les remplace par les siennes. Il établit ainsi deux sessions AES-GCM distinctes : une partagée avec Client 1, une autre avec Client 2.

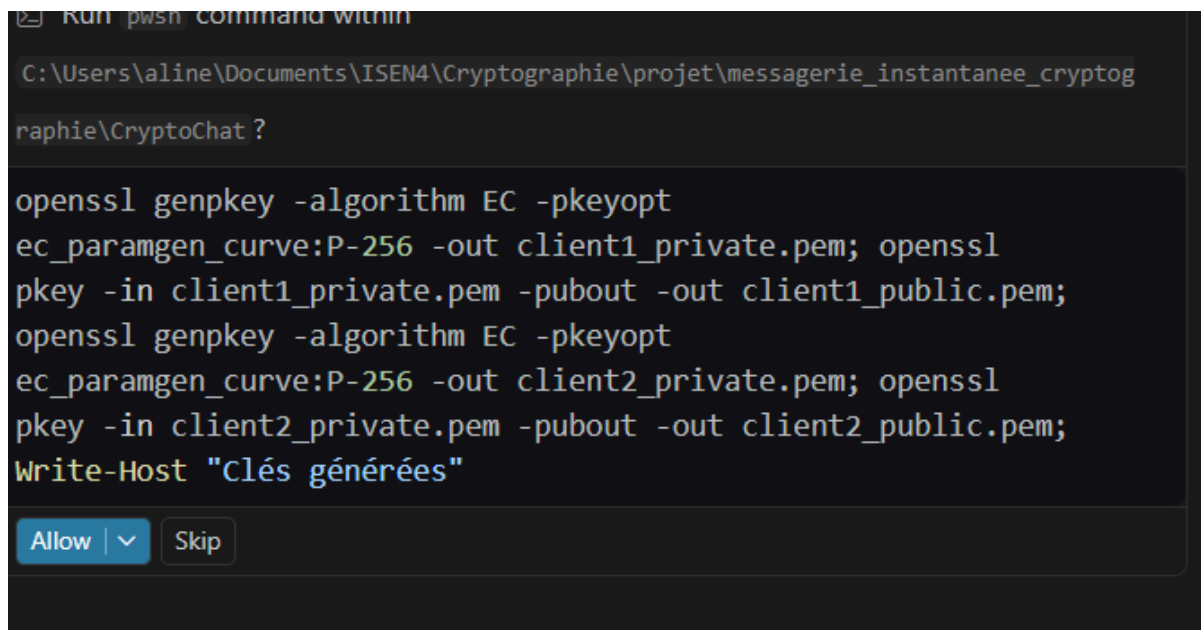
Chaque message reçu est déchiffré avec la clé de la première session, lu en clair en console, puis re-chiffré avec la clé de la seconde avant transmission. Les deux clients croient communiquer de façon sécurisée alors que le serveur lit l'intégralité des échanges. Cette attaque démontre que le chiffrement seul ne suffit pas : sans authentification des clés publiques échangées pendant le handshake, l'échange ECDH est vulnérable à la substitution de clés.



6. Ajout de la signature des clés publiques ECDH

6.1 Génération d'une paire de clé ECDSA long terme

A l'aide d'openssl, nous avons généré une paire de clé ECDSA long terme pour chaque client. La paire de clé du client est passée en paramètre au démarrage :



```

C:\Users\aline\Documents\ISEN4\Cryptographie\projet\messagerie_instantanee_cryptog
raphie\CryptoChat ?

openssl genpkey -algorithm EC -pkeyopt
ec_paramgen_curve:P-256 -out client1_private.pem; openssl
pkey -in client1_private.pem -pubout -out client1_public.pem;
openssl genpkey -algorithm EC -pkeyopt
ec_paramgen_curve:P-256 -out client2_private.pem; openssl
pkey -in client2_private.pem -pubout -out client2_public.pem;
Write-Host "Clés générées"
  
```

6.2 Modification de la phase de handshake

Cette modification consiste à signer la clé publique ECDH éphémère avec la clé privée ECDSA long terme du client.

Concrètement, chaque client envoie désormais :

sa clé publique ECDH + une signature ECDSA de cette clé

À la réception, le client distant utilise la clé publique ECDSA correspondante pour vérifier la signature.

Cette étape permet de garantir que la clé ECDH reçue provient bien du détenteur de la clé privée ECDSA, assurant ainsi l'authenticité de la clé échangée.

```
Crypto Chat - Client 1 - java C X + v
```

```
Connecting to server at localhost:8888...  
Connected to server successfully!  
  
Waiting for other client to connect...  
Both clients connected!  
  
--- Handshake Phase ---  
[Interceptor] Starting ECDH+ECDSA handshake  
[Interceptor] Clé ECDH signée et envoyée  
[Interceptor] Signature ECDSA verifiée OK  
[Interceptor] Cle de session dérivée, handshake terminé !  
--- Handshake Complete ---  
  
Chat session started!  
Type your messages and press Enter to send.  
Type 'exit' to quit.  
  
[Interceptor] Decrypting message (AES-GCM)...  
Other: ah  
salle cyberrrrrrrrrrrrrrr  
[Interceptor] Encrypting message (AES-GCM): salle cyberrrrrrrrrrrrrrr  
You: salle cyberrrrrrrrrrrrrrr  
[Interceptor] Decrypting message (AES-GCM)...  
Other: oui je sais  
[Interceptor] Decrypting message (AES-GCM)...  
Other: neristaaa  
cyndelllllee  
[Interceptor] Encrypting message (AES-GCM): cyndelllllee  
You: cyndelllllee
```

6.3 Implémentation d'une attaque MitM

Malgré l'ajout de la signature des clés publiques ECDH, une attaque Man-in-the-Middle reste possible. En effet, bien que la signature garantisse qu'une clé a été signée par un détenteur de clé privée ECDSA, elle ne garantit pas l'identité réelle du signataire.

Le serveur attaquant peut générer ses propres paires de clés. Il signe ensuite ses propres clés ECDH avec sa propre clé ECDSA et les transmet aux clients. Comme les clients ne disposent d'aucun moyen de vérifier que la clé ECDSA utilisée appartient réellement à l'autre client (absence de PKI), ils acceptent la clé comme valide.

L'attaquant peut alors établir deux sessions distinctes et intercepter les messages, exactement comme dans l'attaque précédente.

Cette étape montre que la signature seule ne suffit pas : il est nécessaire de lier une clé à une identité vérifiable.


```

Crypto Chat - Server - java Se  × + ▾
0UkKUG==
[MITM] Clé de session K1 dérivée avec Client 1
[MITM] Fausse clé ECDH+ECDSA envoyée au Client 2 (signature valide mais clé inconnue)
[Server -> Client 2]: MFkwEwYHk0ZiZj0CAQYIKoZiZj0DAQcDQgAE6Zpk8RLiY2VRqq2c03+b69Ba7gstVSeSBjEi8Y0pDVranTHS0Hfr60IiEtQJnN
sr2sUoddpAE7NfWlBoVd0aRQ==|MEUCIG6gQ0snz/i1eub1GgQMb2SbPiBTpTticjfbX3I8jJw9NAiEA10AMyu3Yb0c9ILB7UfdSolXT1aRkzB18xTL/ith5H
Wk=|MFkwEwYHk0ZiZj0CAQYIKoZiZj0DAQcDQgAETPLCvvr4LXunQXrE6WwQ03wpXRUBCXVsyXk0vrq0vp12E05d4U1cFbd2Yw6UEtFwD0HR3qjGXuU/ftrw
BX1ED0==
[MITM] Interception handshake du Client 2
[MITM] Clé de session K2 dérivée avec Client 2
[MITM] Fausse clé ECDH+ECDSA envoyée au Client 1 (signature valide mais clé inconnue)
[Server -> Client 1]: MFkwEwYHk0ZiZj0CAQYIKoZiZj0DAQcDQgAEyb4R9LksZp3fhHd5d3Epr5uTdpLjsgEpPPccTpVPIJ8SEwWXXs3H64VwVJTV
IQpNdY3pL477JZ+/2vtihFVSA==|MEUCICQpc2yuhH5vciCPXNABpWdDTCi+N9usC2+/M3+0ZgtAiEA8Ao7r7XyVUXjUfzCwIjTgp08I1W68sxV9AZk0gYP
C8=|MFkwEwYHk0ZiZj0CAQYIKoZiZj0DAQcDQgAEQmVB62XznYXBAYIJB5JpJ80+Uk2bTRtZrXTCNYZI+QrmrcqYt77h1GTPBK9Nrr7294BALVYxrro9MB0
362hZ0==
[Client 2 -> Server]: UdpPySFxThx4suXk7a3LmagC0qPFVUu/tmdcCu9p
[MITM] Message en clair intercepté : ah
[Server -> Client 1]: i6fDPpgbInHMeLFC8ORAIxXkWDziUcSH9+D2TCZU
[Client 1 -> Server]: akmaRkPFk2XN+D6v2diUakfqeN9w+WtmC1wJOvpP7+VzBPp9cksh/pBtv9U9xpfbgLGU
[MITM] Message en clair intercepté : salle cyberrrrrrrrrrrrrr
[Server -> Client 2]: 74yAn32DwGtze04ednLTVxc/fgVlQrAK0RRvau8yh+ubf1UurUWKRRRCJCbaH1BPqUE2a
[Client 2 -> Server]: CVQUn3TnQzGmHe9GQ8ar320fK5fGI6bi00p8XTPqcsVYVVIK/
[MITM] Message en clair intercepté : oui je sais
[Server -> Client 1]: Ek70byGDIq/q4WVxZosg2SyF/o3lzwLlQud1/iL41P83NCg1czP
[Client 2 -> Server]: SIBoUsOkDbRYPLrBImiJw80JirzFJZySUr70KCXwuRc5L+OzQ==
[MITM] Message en clair intercepté : neristaaa
[Server -> Client 1]: vDK8eywiYXHP+l8xAlYwV3fs78P9S29rcShZmRqXDhPpPpbMnA==
[Client 1 -> Server]: luWib+yFArA/aEGAKoLXDQ/AlkJFwAz61ncbXTUmy4vvNWzkYxW
[MITM] Message en clair intercepté : cyndelllle
[Server -> Client 2]: vx4k4KZ4+jJda8/JVP9LX+xYZpyorlQdh/TY1uvbMzPejsJao8w1

```

7. Certification des clés

Nous avons vu précédemment que la signature ECDSA prouvait qu'une clé a bien été signée par son détenteur, mais sans permettre d'en vérifier l'identité. Or, cette partie introduit une Infrastructure à Clé Publique (PKI) : une Autorité de Certification (CA) tierce signe les clés publiques des clients, créant une chaîne de confiance vérifiable.

7.1 Génération de la PKI

La CA est créée à l'aide de deux commandes OpenSSL : `openssl genpkey` génère une clé privée EC sur P-256, puis `openssl req -x509` produit un certificat auto-signé. Ce certificat constitue l'ancrage de confiance (trust anchor). Les certificats clients sont générés via une CSR (Certificate Signing Request) signée par la CA, liant l'identité `CN=Client1` ou `CN=Client2` à la clé publique ECDSA correspondante.

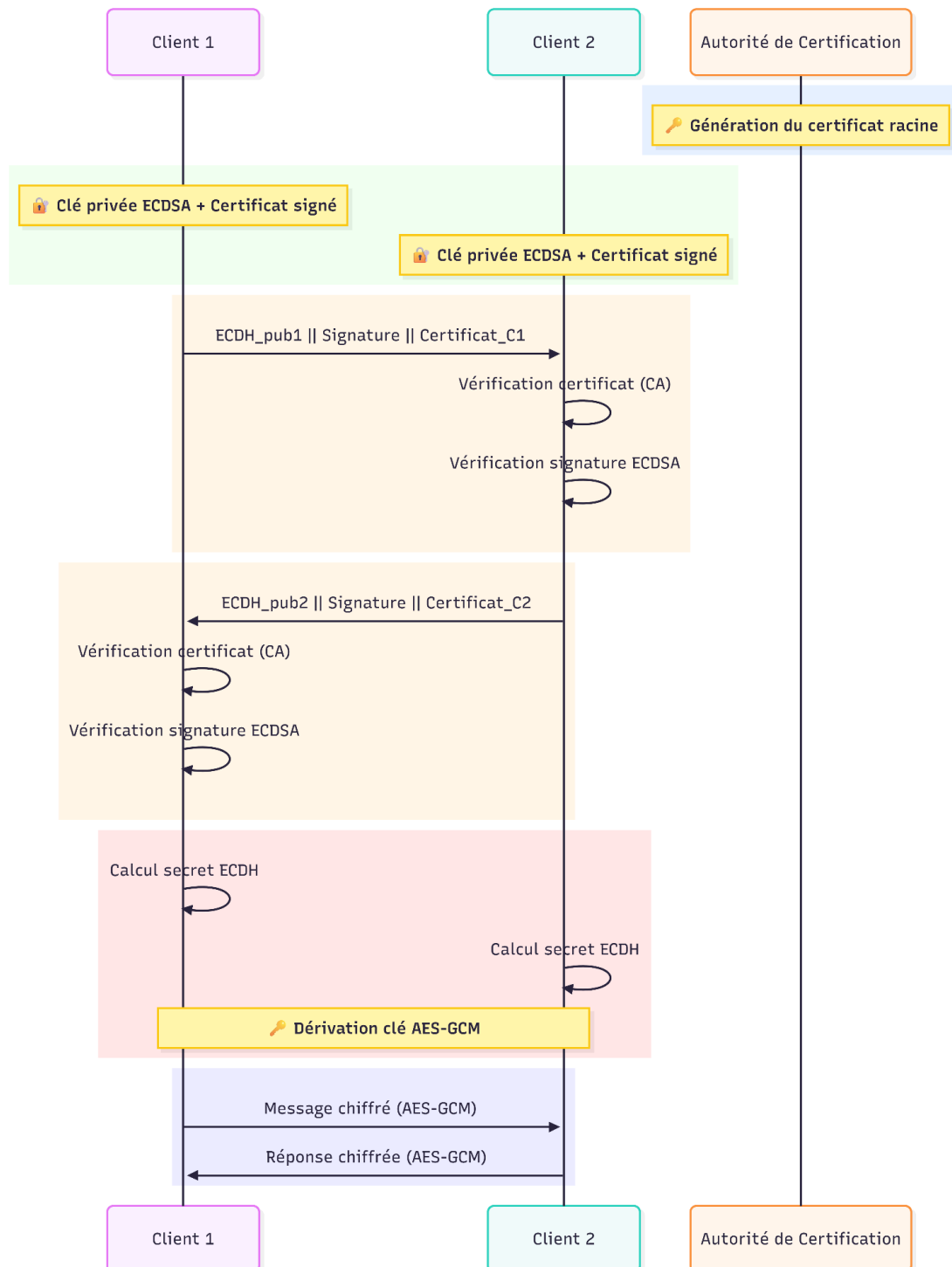
7.2 Mise à jour du protocole de handshake

Chaque client envoie désormais : `Base64(ecdhPub) | Base64(sig) | Base64(cert_DER)`. Le certificat X.509 encodé DER remplace la clé ECDSA brute. À la réception, le client parse le certificat (`CertificateFactory`), vérifie sa signature via `otherCert.verify(caCert.getPublicKey())`, extrait la clé certifiée, puis vérifie la signature ECDSA sur la clé ECDH éphémère. La chaîne est complète : CA certifie l'identité, l'identité certifie la clé ECDH, la clé ECDH établit la session AES-GCM.

7.3 Détection du MitM

Le serveur MitM ne possède pas `ca_private.pem` et ne peut donc pas émettre de certificat X.509 valide. Lorsqu'il tente de transmettre sa clé brute à la place d'un certificat, le parsing côté client échoue avec "Unable to initialize: Too short". La connexion est refusée. Cette erreur confirme que la PKI protège efficacement contre l'usurpation d'identité, à condition que `ca_private.pem` reste protégée.

Voici un schéma pour illustrer cette logique :



```
Crypto Chat - Server
[Server] WITH PKI mode - tentative de forge de certificats
=== Crypto Chat Server ===
Starting server on port 8888...
Server started successfully!
Waiting for 2 clients to connect...

Client 1 connected from 127.0.0.1
Client 2 connected from 127.0.0.1

All clients connected! Chat session can begin.

[Server -> Client 1]: READY
[Server -> Client 2]: READY
Server is now relaying messages between clients...
[Client 2 -> Server]: MFkwEwYHkoZiZj0CAQYIKoZiZj0DAQcDQgAE/alt4Hf9Ygw15jZBfVU5vTiVBDZSSiKvbyMySMDlQYz/taJtRZ1AYAPzqgc6gl/z9Fsfwj3UuQ3msZ/d9IFiVg==|MEQCID4KS
smSunip6ZukbMr15FRZFZPmWJ/oOk03tEbuMKUATiAoyjxjRZ83sW4B1fwfVZmwR/vRx44WwTmo9Pbcb91tRA==|MIIBzCCARSAwIBAglUeInnhyrJ89U/vcKlwTjkiYRYLFwmcGyYIKoZiZj0EAWIwGDEW
MBQGA1UEAwNQ3JScHRvQ2hhdCBDQTAEfW0yNjA0MDg0NjAxMzhaMBIxEDAOBgNVBAMMB0NsaWVudDIwMTATBgqhkhjOPQMBBwNCAAR1YmJqKqccUK81ph+D/CTr
Mqo+41QNHyT23KnrKSmck2W4MBxrignsNd0aAYpiad3MuRjXzbD5r5HD9FRSQpSo0IwQDAdBgNVHQ4EfgQUZ36rg/Qm9GtuLmeo/tZPRDuyhH8WwYDVR0jBBgwFoAUcxiy6WdKj0pS0FTVcsWf8rLqrbgw
CgYIKoZiZj0EAWIDRwAwRAIgaDA8bUUAxj9RVAQ/w415c5K4jVNP8pZ6T8ZdJmRUCIfjZ7d8J9ovp+GtQEdE1DF7Ij9Qo7f7vqpDtmYlHWqIH
[Client 1 -> Server]: MFkwEwYHkoZiZj0CAQYIKoZiZj0DAQcDQgAEeh6sn96hTA1S9Y1uGULfLve0MRTnsuzI4WvnaHScVBpGm1mtXuvWY/WoqJpnnvDlnDsKQW4M4QvQ3Cz7rqCQ==|HEUCICKFF
wFQZ0fy/VNqKVGq8LIZI SUPKDVUwJ765x8S01PgAIEA2dq4pcNPysbRjvNGQqTVEWAN/atbVWaxEfpf6hVMH8G==|MIIBzCCARSAwIBAglUeInnhyrJ89U/vcKlwTjkiYRYLFwmcGyYIKoZiZj0EAWIwGDEW
MBQGA1UEAwNQ3JScHRvQ2hhdCBDQTAEfW0yNjA0MDg0NjAxMzhaMBIxEDAOBgNVBAMMB0NsaWVudDIwMTATBgqhkhjOPQMBBwNCAAR1YmJqKqccUK81ph+D/CTr
Mqo+41QNHyT23KnrKSmck2W4MBxrignsNd0aAYpiad3MuRjXzbD5r5HD9FRSQpSo0IwQDAdBgNVHQ4EfgQUZ36rg/Qm9GtuLmeo/tZPRDuyhH8WwYDVR0jBBgwFoAUcxiy6WdKj0pS0FTVcsWf8rLqrbgw
CgYIKoZiZj0EAWIDRwAwRAIgaDA8bUUAxj9RVAQ/w415c5K4jVNP8pZ6T8ZdJmRUCIfjZ7d8J9ovp+GtQEdE1DF7Ij9Qo7f7vqpDtmYlHWqIH
[MITM] Interception handshake du Client 2
[MITM] Clé de session K2 dérivée avec Client 2
[MITM] Faux 'certificat' envoyé au Client 1 - pas signé par la CA, détection imminente !
[Server -> Client 1]: MFkwEwYHkoZiZj0CAQYIKoZiZj0DAQcDQgAEtKkKjV9Wk/zr/4B17bJ9orevS34H/VHz/v5JwqqkYp/LC99ExpK7T02ih2NZx8Q2oZ+Hebo+dp4DLOYhou1Q==|MEQCIG78j
xIB7R6GfKvZpZpVteB4gBgkRRZC+KwOguPGPjBqAiBm0ZezwRvUsJrdV2YRzt+mo9s6C7iDNcmYyJ3CWZfjyQ==|MFkwEwYHkoZiZj0CAQYIKoZiZj0DAQcDQgAEfXa4JeHy+AM2+PYoqcSEQKc/29Nx2+j
s7rv/Fdd1xwbkzmgBOM7s1yBd3dNt/j1wGo0vRCckYLnVdph8tUQ==
[MITM] Interception handshake du Client 1
[MITM] Clé de session K1 dérivée avec Client 1
[MITM] Faux 'certificat' envoyé au Client 2 - pas signé par la CA, détection imminente !
[Server -> Client 2]: MFkwEwYHkoZiZj0CAQYIKoZiZj0DAQcDQgAE88FTwN25ZpdGbUe1Veo1GASTE8D+jPL6DnSAnT6PmT/GlZrve9HQ5VXfRw1qRJRJWS2960sz0aW1YHMaSTMQ==|MEUCIGH8a
yNeAJulg22zP2DQ2C4u1uXcjJmCtHOF19NTWQmAiEA8qL/JvP5mlavJ9bp3q9+Ap1lbcAsj31LEbq3WSvOkQ=|MFkwEwYHkoZiZj0CAQYIKoZiZj0DAQcDQgAEdVVFfA08q++q/ZaejLpY8Viam/hHwqVkt
M+EAqTgvPrSnsH1TVexMgU5Q0i0SEQeEnKovSjKabb7QK1r8WdW0A==
Client 1 disconnected. Active clients: 1
Client 2 disconnected. Active clients: 0

C:\Users\aline\Documents\ISEN4\Cryptographie\projet\messagerie_instantanee_cryptographie\CryptoChat>
```

Les captures d'écran ci-dessous illustrent le rejet de la connexion MitM, l'affichage de l'identité du client distant (CN=Client1 / CN=Client2) et la session sécurisée établie avec succès entre les deux clients légitimes.

```
Crypto Chat - Client 2
Starting client ...
[Interceptor] Certificat chargé : CN=Client2
[Interceptor] CA : CN=CryptoChat CA
=== Crypto Chat Client ===
Connecting to server at localhost:8888...
Connected to server successfully!

Waiting for other client to connect...
Both clients connected!

--- Handshake Phase ---
[Interceptor] Starting ECDH+PKI handshake
[Interceptor] Clé ECDH signée + certificat envoyés
Connection error: [ERREUR PKI] Certificat invalide - attaque MitM detectee ! Unable to initialize, java.io.IOException:
Too short
java.io.IOException: [ERREUR PKI] Certificat invalide - attaque MitM detectee ! Unable to initialize, java.io.IOExceptio
n: Too short
    at Interceptor.onHandshake(Interceptor.java:74)
    at Client.start(Client.java:52)
    at Client.main(Client.java:29)
Disconnected from server.

C:\Users\aline\Documents\ISEN4\Cryptographie\projet\messagerie_instantanee_cryptographie\CryptoChat>
```

```
Crypto Chat - Client 1
Starting client ...
[Interceptor] Certificat chargé : CN=Client1
[Interceptor] CA : CN=CryptoChat CA
=== Crypto Chat Client ===
Connecting to server at localhost:8888...
Connected to server successfully!

Waiting for other client to connect...
Both clients connected!

--- Handshake Phase ---
[Interceptor] Starting ECDH+PKI handshake
[Interceptor] Clé ECDH signée + certificat envoyés
Connection error: [ERREUR PKI] Certificat invalide - attaque MitM detectee ! Unable to initialize, java.io.IOException:
Too short
java.io.IOException: [ERREUR PKI] Certificat invalide - attaque MitM detectee ! Unable to initialize, java.io.IOExceptio
n: Too short
    at Interceptor.onHandshake(Interceptor.java:74)
    at Client.start(Client.java:52)
    at Client.main(Client.java:29)
Disconnected from server.

C:\Users\aline\Documents\ISEN4\Cryptographie\projet\messagerie_instantanee_cryptographie\CryptoChat>
```

8. Attaques résiduelles

8.1 Attaques encore possibles

Malgré la mise en place d'une PKI complète, certaines attaques restent possibles au niveau du transport des messages.

Attaque par jeu (Replay attack)

Malgré l'utilisation d'AES-GCM et d'une PKI, certaines attaques restent possibles car ces mécanismes ne garantissent pas l'unicité des messages. Dans cette attaque, le serveur malveillant enregistre un message chiffré valide envoyé par un client. Ensuite, lors d'un nouvel envoi, il remplace le message courant par un ancien message précédemment enregistré.

```
Crypto Chat - Server - java Se x + v
6WDkJ0pS0FTVcsWf8rLqrbgwCgYIKoZIzj0EAwIDSQAwwRgIhAKG25yITN29Y2FM6aQW8SSKmplkXmLki5EJC4RNhpEPfAiEAvpKvz4GgErMDBQq22JyGefg
xq7fLJBBeVSn8awkwBhQ=
[Client 1 -> Server]: MFkwEwYHkoZIzj0CAQYIKoZIzj0DAQcDQgAEduMmSpXPpmrgD5EHhNQesMdUCMxL03Km9HwK0dSBm8sJmTG187GwOI2PW2rbsZ
MitJ0R96svKvncexAPI0QgA==MEUCIBMwcd9uzux7SF3/DAMB3x/pvFQp/XXx4ge8sV1Hpx/RAiEaiYOn/ZqcNc2DmIciUvW2b754LU/W+GxduwsNhwoD6
Vk=MIIBbTCCARsgAwIBAgIUeINhyrJ89U/vcKlwTjkIyRYLfwCgYIKoZIzj0EAwIwGDEWMBQGA1UEAwNQ3J5cHRvQ2hhdBDBQTAeFw0yNjA0MDg0xNjAx
MzhaFw0yNjA0MDg0xNjAxMzhaMBIxEDAOBgNVBAMMB0NsaWVudDEWWTATBgqhkhjOPQIBBggqhkhjOPQMBBwNCAAAQzgWJu2bm1ub4ghaqKyjbEhIB8rXn1aJKL
Xie+YWEIztnf2vXwDso00s2TDLK5UAP/K/0PRcM71WIUrAaZ7qZfo0IwQDAdBgNVHQ4EFgQUUM3YnmQ/VYxcm/FRUBoIHL3dToIwHwYDVR0jBBgwFoAUcxiy
6WDkJ0pS0FTVcsWf8rLqrbgwCgYIKoZIzj0EAwIDRwAwRAIgaDA0bmUAxj9RVAq/w415c5K4jVNPLH8pz6TT0ZdJmRUCIFjZ7d8J9ovp+GtQEdE1DF7IjgQo
7f7vqpDTmYLHWqIM
[Server] Relaying handshake from Client 1
[Server -> Client 2]: MFkwEwYHkoZIzj0CAQYIKoZIzj0DAQcDQgAEduMmSpXPpmrgD5EHhNQesMdUCMxL03Km9HwK0dSBm8sJmTG187GwOI2PW2rbsZ
MitJ0R96svKvncexAPI0QgA==MEUCIBMwcd9uzux7SF3/DAMB3x/pvFQp/XXx4ge8sV1Hpx/RAiEaiYOn/ZqcNc2DmIciUvW2b754LU/W+GxduwsNhwoD6
Vk=MIIBbTCCARsgAwIBAgIUeINhyrJ89U/vcKlwTjkIyRYLfwCgYIKoZIzj0EAwIwGDEWMBQGA1UEAwNQ3J5cHRvQ2hhdBDBQTAeFw0yNjA0MDg0xNjAx
MzhaFw0yNjA0MDg0xNjAxMzhaMBIxEDAOBgNVBAMMB0NsaWVudDEWWTATBgqhkhjOPQIBBggqhkhjOPQMBBwNCAAAQzgWJu2bm1ub4ghaqKyjbEhIB8rXn1aJKL
Xie+YWEIztnf2vXwDso00s2TDLK5UAP/K/0PRcM71WIUrAaZ7qZfo0IwQDAdBgNVHQ4EFgQUUM3YnmQ/VYxcm/FRUBoIHL3dToIwHwYDVR0jBBgwFoAUcxiy
6WDkJ0pS0FTVcsWf8rLqrbgwCgYIKoZIzj0EAwIDRwAwRAIgaDA0bmUAxj9RVAq/w415c5K4jVNPLH8pz6TT0ZdJmRUCIFjZ7d8J9ovp+GtQEdE1DF7IjgQo
7f7vqpDTmYLHWqIM
[Client 2 -> Server]: wxc6qcQPYdkXFp5mX0r805kbEWSVUEuvhsU8k4dYdHoB
[Server -> Client 1]: relay normal
[Server -> Client 1]: wxc6qcQPYdkXFp5mX0r805kbEWSVUEuvhsU8k4dYdHoB
[Client 2 -> Server]: Mk7xmEkRIWsm+00j2DFCL7ycDD5XQN7k6ydwG45tNUGdy092kQ/KHhA8ZxbzJGRdXy5A0Hcjf66/ZqQJ0eB2IA==
[Server -> Client 1]: relay normal
[Server -> Client 1]: Mk7xmEkRIWsm+00j2DFCL7ycDD5XQN7k6ydwG45tNUGdy092kQ/KHhA8ZxbzJGRdXy5A0Hcjf66/ZqQJ0eB2IA==
[Client 1 -> Server]: uZ3ypXQHvbBPhGJJ/ea6hB9oCfAK+XRB7XVcSQNgqA==
[Server] 1er message Client 1 sauvegarde (seq=0), relay normal
[Server -> Client 2]: uZ3ypXQHvbBPhGJJ/ea6hB9oCfAK+XRB7XVcSQNgqA==
[Client 1 -> Server]: 2sEcFPUQRVgq9hpQT0F26d0PEXc903twGqg18nIM0qwcMjohai6Qr2jncvk=
[Replay] ATTAQUE REJEU : seq=0 rejoue vers Client 2 (attendu seq=1) !
[Server -> Client 2]: uZ3ypXQHvbBPhGJJ/ea6hB9oCfAK+XRB7XVcSQNgqA==
```

```
Crypto Chat - Client 1 - java C x + v
=== Crypto Chat Client ===
Connecting to server at localhost:8888...
Connected to server successfully!

Waiting for other client to connect...
Both clients connected!

--- Handshake Phase ---
[Interceptor] Starting ECDH+PKI handshake
[Interceptor] Clé ECDH signée + certificat envoyés
[Interceptor] Certificat verifie OK : CN=Client2
[Interceptor] Signature ECDSA verifiee OK
[Interceptor] Cle de session derivee, handshake termine !
--- Handshake Complete ---

Chat session started!
Type your messages and press Enter to send.
Type 'exit' to quit.

[Interceptor] Decrypting message (AES-GCM, sans seq)...
Other: salut
[Interceptor] Decrypting message (AES-GCM, sans seq)...
Other: on envoie le message sans protection
oui
[Interceptor] Encrypting message (AES-GCM, sans seq): oui
You: oui
deuxieme message
[Interceptor] Encrypting message (AES-GCM, sans seq): deuxieme message
You: deuxieme message
```

Attaque par suppression

Un attaquant peut choisir de ne pas relayer certains messages.

Le client ne peut pas distinguer une suppression malveillante d'un problème réseau.

8.2 Mécanisme de protection

Afin de se protéger contre les attaques de rejeu et de suppression, un mécanisme de numérotation des messages a été introduit. Ce mécanisme repose sur l'ajout d'un numéro de séquence unique à chaque message échangé entre les clients. Concrètement, avant le chiffrement, chaque message est précédé d'un compteur incrémental. Ce compteur est initialisé au début de la session puis augmenté de 1 à chaque envoi. Le numéro de séquence est concaténé au message en clair, puis l'ensemble est chiffré avec AES-GCM.

L'utilisation d'AES-GCM est essentielle, car le tag d'authentification couvre à la fois le contenu du message et le numéro de séquence. Toute modification du numéro ou du message entraîne donc une erreur de vérification et le message est rejeté.

À la réception, le client extrait le numéro de séquence après déchiffrement et le compare à la valeur attendue. Si le numéro reçu est inférieur à celui attendu, cela signifie qu'un ancien message a été rejoué. À l'inverse, si un saut est détecté dans la séquence, cela indique qu'un message a été supprimé.

Ce mécanisme permet ainsi de détecter :

- les attaques par rejeu, en refusant les messages déjà vus
- les attaques par suppression, en identifiant les trous dans la séquence

Il renforce le protocole en ajoutant une notion d'ordre et de continuité des messages, qui n'est pas assurée par le chiffrement seul.

8.3 Perfect Forward Secrecy

Un attaquant qui enregistre les messages chiffrés puis obtient plus tard les clés privées ECDSA ne peut pas déchiffrer les anciennes communications.

En effet, les clés de session AES sont dérivées via ECDH éphémère. Ces clés sont temporaires et supprimées après usage. Puis, les clés ECDSA servent uniquement à l'authentification

Cette propriété est appelée : **Perfect Forward Secrecy (PFS)**

9. Difficultés rencontrées

La principale difficulté rencontrée a été la prise en main du langage Java, que notre groupe pratique peu. Par exemple, nous ne connaissons pas la nécessité de compiler les fichiers sources avec `javac *.java` avant l'exécution, ni le rôle des fichiers `.class` générés.

La gestion des types de clés Java (`PrivateKey`, `PublicKey`, `SecretKey`) et les conversions entre formats PEM, DER et objets Java (`PKCS8EncodedKeySpec`, `X509EncodedKeySpec`) ont nécessité un temps d'adaptation important. La compréhension du masque `& 0xFF` pour la reconstruction d'entiers depuis des bytes signés a également demandé de l'attention.

La gestion de l'IV en AES-GCM a aussi posé des questions : il fallait comprendre pourquoi réutiliser un IV avec la même clé est une faille critique, et pourquoi la taille recommandée est 12 octets et non 16. La phase de handshake a nécessité une synchronisation précise entre les deux clients pour s'assurer qu'ils échangent bien leurs clés avant tout envoi de message.

En pratique, la compréhension et la manipulation de l'API cryptographique Java (JCE) ont représenté un frein important. Une grande partie du temps a été consacrée à comprendre le fonctionnement du code (gestion des clés, formats, API) plutôt que les concepts cryptographiques eux-mêmes. Cette situation a été une source de frustration, car les erreurs provenaient souvent de détails d'implémentation (types, encodage, exceptions) et non des principes de sécurité étudiés.

Paradoxalement, les concepts cryptographiques se sont révélés plus simples à assimiler que leur mise en œuvre concrète en Java.

10. Améliorations possibles du projet

Plusieurs améliorations pourraient être apportées au projet.

Tout d'abord, la dérivation de clé à partir d'un mot de passe pourrait être améliorée. L'utilisation directe de SHA-256 est simple à implémenter mais peu sécurisée en pratique. Des fonctions comme PBKDF2 ou Argon2 permettraient de mieux résister aux attaques par brute-force, même si leur mise en œuvre aurait été plus complexe pour ce projet.

Ensuite, la gestion des certificats reste très simplifiée. Dans un système réel, il faudrait gérer la révocation des certificats, leur expiration et leur distribution sécurisée. Ces aspects n'ont pas été implémentés car ils dépassent le cadre du projet mais constituent une étape importante vers un protocole réel.

Enfin, le projet fonctionne uniquement avec deux clients. Une extension intéressante serait de gérer plusieurs utilisateurs simultanément, ce qui introduirait des problématiques supplémentaires comme la gestion de clés de groupe.

11. Ressources utilisées

- ROT13 ([description wikipédia](#))
- AES ([description wikipédia](#))
- Base64 ([description wikipédia](#))
- GCM ([description wikipédia](#))
- AES-GCM ([discussion Reddit](#))
- ECDH / Diffie-Hellman ([description wikipédia](#))
- ECDSA ([description wikipédia](#))
- Certificats X.509 ([description wikipédia](#))
- Perfect Forward Secrecy ([description wikipédia](#))
- OpenSSL — génération de certificats ([documentation officielle](#))
- Java Cryptography Architecture (JCA) ([documentation Oracle](#))
- ChatGPT pour la génération de Mermaid Charts

12. Conclusion

Ce projet a permis de parcourir l'ensemble des couches d'un protocole de communication sécurisé moderne, depuis le simple ROT13 jusqu'à une PKI complète avec Perfect Forward Secrecy. Chaque étape a montré concrètement pourquoi un mécanisme seul est insuffisant et comment les attaques évoluent en réponse aux protections mises en place.

De plus, il a été particulièrement formateur dans un contexte d'élève ingénieur. Il a permis de comprendre que la sécurité ne repose pas sur un seul algorithme, mais sur l'assemblage cohérent de plusieurs mécanismes.

Il met également en évidence l'écart entre la théorie et l'implémentation : un protocole correct sur le papier peut devenir vulnérable à cause d'erreurs techniques ou de mauvaises hypothèses.

Enfin, ce projet donne une vision concrète des protocoles utilisés en pratique et permet de mieux comprendre les enjeux réels de la cybersécurité dans les systèmes distribués.